

¿Qué son los Hooks de Rendimiento? ¿Cuáles son sus características y para qué sirve? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descrito, explicando el ejemplo.

Los **Hooks de rendimiento** son herramientas diseñadas para evitar cálculos repetitivos innecesarios y prevenir que los componentes se vuelvan a procesar si sus datos no cambiaron real y visualmente.

Las características centrales de estos Hooks son que guardan en la memoria el resultado de una operación (como hace useMemo) o la definición exacta de una función (como hace useCallback), asociándolos a una lista de dependencias; de este modo, React reutiliza lo que ya guardó en memoria y solo vuelve a calcularlo si alguna de esas dependencias específicas cambia de valor. Sirven principalmente para mantener la aplicación rápida, fluida y sin trabas visuales cuando se procesan grandes volúmenes de información en la interfaz.

```
import React, { useState, useMemo } from 'react';

function FiltroPalabras() {
  // Estado para el término de búsqueda tipeado
  const [busqueda, setBusqueda] = useState('');
  // Estado independiente para un contador simple
  const [contador, setContador] = useState(0);

  // Lista fija de elementos simulada
  const palabras = ['React', 'JavaScript', 'HTML', 'CSS', 'Node', 'Vite'];

  // Hook useMemo: Guarda el resultado del filtrado en memoria.
  // Solo se vuelve a ejecutar la búsqueda si el texto 'busqueda' cambia,
  // evitando procesar la lista cuando cambia el estado 'contador'.
  const palabrasFiltradas = useMemo(() => {
    return palabras.filter((palabra) =>
      palabra.toLowerCase().includes(busqueda.toLowerCase())
    );
  }, [busqueda]);

  return (
    <div>
      { /* Campo de texto que modifica la dependencia de useMemo */ }
      <input
        type="text"
        placeholder="Buscar tecnología..."
        value={busqueda}
        onChange={(e) => setBusqueda(e.target.value)}
      />

      { /* Botón que cambia un estado ajeno al filtro */ }
      <button onClick={() => setContador(contador + 1)}>
        Clics: {contador}
      </button>

      { /* Renderizado de la lista optimizada */ }
      <div>
        {palabrasFiltradas.map((item, index) => (
          <p key={index}>{item}</p>
        ))}
      </div>
    </div>
  );
}

export default FiltroPalabras;
```

**¿Qué es el contexto en React? ¿Cuáles son sus características y para qué sirve?
¿Qué diferencia existe con los props? Desarrollar un ejemplo sencillo en código REACT que permita visualizar lo descrito, explicando el ejemplo.**

El **contexto** es un mecanismo que permite compartir datos de forma directa y global a cualquier componente del árbol visual, sin necesidad de pasarlos manualmente por cada nivel intermedio.

Su característica esencial es la comunicación directa: se define un proveedor (Context.Provider) en un componente superior con los datos a compartir, y cualquier componente hijo ubicado más abajo puede leer esa información inmediatamente usando el hook useContext.

La gran diferencia frente a los *props* tradicionales es que las props viajan obligatoriamente de forma escalonada de padre a hijo directo; si un componente ubicado en el fondo necesita un dato, las props deben atravesar componentes intermedios que no necesitan esa información, un problema común conocido como perforación de propiedades (*prop drilling*).

El contexto sirve para centralizar información compartida por toda la aplicación, como el usuario que ha iniciado sesión, el idioma seleccionado o el tema visual (modo claro u oscuro).

Ejemplo en la próxima hoja.

```

import React, { createContext, useContext, useState } from 'react';

// 1. Creación del contexto global para el tema visual
const TemaContexto = createContext();

// Componente nieta que necesita consumir el dato directamente
function BotonTema() {
  // Hook useContext: Lee el valor compartido sin recibirlo por props
  const { tema, alternarTema } = useContext(TemaContexto);

  return (
    <button onClick={alternarTema}>
      Modo actual: {tema} (Toca para cambiar)
    </button>
  );
}

// Componente intermedio que no usa el dato y no pasa props
function BarraNavegacion() {
  return (
    <nav>
      <p>Menú Principal</p>
      {/* Inserta el botón sin tener que enviarle propiedades */}
      <BotonTema />
    </nav>
  );
}

// Componente principal que suministra la información
export default function App() {
  const [tema, setTema] = useState('claro');

  const alternarTema = () => {
    setTema(tema === 'claro' ? 'oscuro' : 'claro');
  };

  return (
    // 2. El Provider envuelve a los hijos y les comparte el valor global
    <TemaContexto.Provider value={{ tema, alternarTema }}>
      <div style={{ padding: '20px' }}>
        <h1>Sitio Web</h1>
        <BarraNavegacion />
      </div>
    </TemaContexto.Provider>
  );
}

```